
Software Requirements Specification

for

Multi-Agentive RAG Research System

Version 1.0

Prepared by

Group Number: 44

Mude Sai Abhinav Chauhan
Anish Boppidi Reddy
Aishwarya Vodnala
Vinay Patil
Tejas Pothu
Siddharth Uppala
Supreeth Kunaparaju
Sujato Dutta

SE23UCSE198
SE23UCSE042
SE23UCSE187
SE23UCSE186
SE23UCSE175
SE23UCSE241
SE23UCSE098
SE23UARI167

saiabhinavchauhan@gmail.com
anishreddyboppidi@gmail.com
aishwarya28.vodnala@gmail.com
vinay25004@gmail.com
tejaspothu@gmail.com
siddharthupl456@gmail.com
supreethkunaparaju@gmail.com
sujatodutta0204@gmail.com

Instructor: Avinash Arun Chauhan

Course: Software Engineering

Lab Section: Ground Floor Lab IT2

Teaching Assistant: Sai Srithaja Nibhabupudi

Date: 09/03/2026

CONTENTS.....	II
REVISIONS.....	II
1 INTRODUCTION.....	1
1.1 DOCUMENT PURPOSE.....	1
1.2 PRODUCT SCOPE.....	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	2
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	2
1.5 DOCUMENT CONVENTIONS.....	2
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	2
2 OVERALL DESCRIPTION.....	3
2.1 PRODUCT OVERVIEW.....	2
2.2 PRODUCT FUNCTIONALITY.....	3
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	3
2.4 ASSUMPTIONS AND DEPENDENCIES.....	4
3 SPECIFIC REQUIREMENTS.....	5
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	5
3.2 FUNCTIONAL REQUIREMENTS.....	8
3.3 USE CASE MODEL.....	10
4 OTHER NON-FUNCTIONAL REQUIREMENTS.....	15
4.1 PERFORMANCE REQUIREMENTS.....	15
4.2 SAFETY AND SECURITY REQUIREMENTS.....	15
4.3 SOFTWARE QUALITY ATTRIBUTES.....	15
5 OTHER REQUIREMENTS.....	16
APPENDIX A – DATA DICTIONARY.....	17
APPENDIX B - GROUP LOG.....	21

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Mude Sai Abhinav Chauhan	Initial draft of the Software Requirements Specification for the Multi-Agentive RAG AI Research Scientist system.	09/03/26

1 Introduction

This document provides the Software Requirements Specification (SRS) for the **Multi-Agentic RAG AI Research Scientist system**. The system is designed to automate research tasks by combining multiple intelligent agents with Retrieval-Augmented Generation (RAG).

The system processes a user's research query, retrieves relevant academic papers from sources such as arXiv, extracts and indexes their content, and produces structured responses supported by citations. The architecture consists of specialized agents that perform tasks such as planning, searching, document ingestion, retrieval, reasoning, and safety validation.

1.1 Document Purpose

The purpose of this document is to define the functional and non-functional requirements of the **Multi-Agentic RAG AI Research Scientist system**. It describes the system's features, architecture, constraints, and expected behavior.

This document serves as a reference for developers, instructors, and evaluators to understand how the system is designed and what requirements must be satisfied during development and evaluation.

1.2 Product Scope

The Multi-Agentic RAG AI Research Scientist is an AI-powered research assistant that automates literature search and analysis.

The system takes a research query from the user and processes it through a pipeline of specialized agents. These agents retrieve academic papers, process their content, perform semantic search, rank relevant information, and synthesize responses supported by citations.

The system provides several benefits:

- Automated academic literature search
- Faster research summarization
- Reduced hallucination through citation verification
- Modular multi-agent architecture for scalable AI workflows

1.3 Intended Audience and Document Overview

This document is intended for:

- **Developers** – to understand system architecture and implementation requirements
- **Course Instructor** – to evaluate the project design and requirements
- **Project Team Members** – to guide development and testing

The document begins with an introduction to the system, followed by an overall description of the product, detailed functional requirements, and non-functional requirements such as performance and security.

1.4 Definitions, Acronyms and Abbreviations

AI – Artificial Intelligence
API – Application Programming Interface
FAISS – Facebook AI Similarity Search
LLM – Large Language Model
RAG – Retrieval Augmented Generation
PDF – Portable Document Format
arXiv – Open access repository of research papers

1.5 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) format. The text uses Arial font size 11 or 12 with single spacing and 1-inch margins as defined in the template. Section headings follow the numbering structure of the template. Functional requirements are written using the format “The system shall ...” to clearly describe system behavior.

1.6 References and Acknowledgments

References used in this project include:

- arXiv API Documentation
- FAISS Documentation
- Research papers on Retrieval-Augmented Generation
- Course materials from the Software Engineering course

2 Overall Description

2.1 Product Overview

The **Multi-Agentic RAG AI Research Scientist** is an AI-based system designed to assist users in performing academic research tasks. The system processes a user's research query using multiple specialized agents that work together to retrieve, analyze, and synthesize information from research papers.

The system integrates with external research sources such as **arXiv** to retrieve relevant academic papers. The retrieved documents are processed and stored in a vector database using **FAISS**, enabling semantic search and retrieval. Different agents in the system handle tasks such as query planning, paper retrieval, document ingestion, ranking, reasoning, and safety validation.

The final output generated by the system is a **structured research response with supporting citations**, helping users quickly understand research topics and relevant literature.

2.2 Product Functionality

The system performs the following major functions:

- Accept research queries from users
- Decompose queries using a Planner Agent
- Retrieve research papers from arXiv using Search Agent
- Download and process PDF documents
- Generate embeddings and store them in a FAISS vector database
- Perform semantic retrieval of relevant information
- Rerank retrieved documents using LLM scoring
- Generate research summaries using reasoning agents
- Validate citations and reduce hallucination using Safety Agent
- Produce structured answers with references

2.3 Design and Implementation Constraints

The development of the Multi-Agentic RAG AI Research Scientist system is subject to several design and implementation constraints. The system must be implemented using Python as the primary programming language and should follow a modular multi-agent architecture where different agents handle specific tasks such as planning, searching, retrieval, and reasoning.

The system relies on external technologies such as the arXiv API for retrieving academic papers and FAISS for vector-based semantic search. Therefore, the design must ensure compatibility with these tools and their libraries.

Additionally, the system must follow the software engineering design principles and modeling practices taught in the course. Development must also consider limitations such as available computing resources, API usage limits, and processing time for handling large research documents.

2.4 Assumptions and Dependencies

The system assumes:

- arXiv API remains accessible and functional
- Internet connectivity is available
- The LLM used for reasoning is accessible through API or local model
- PDF documents are available and properly formatted

Dependencies include:

- FAISS library for vector indexing
- Python libraries for PDF processing
- LLM frameworks for reasoning tasks

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system provides a **web-based user interface** that allows users to submit research queries and view generated research summaries.

Interface Characteristics

The interface consists of:

- Query input box
- Submit/Search button
- Results display panel
- Citation list panel

Users interact through a **keyboard and mouse** using a standard web browser.

The UI displays:

- Research summary generated by the reasoning agent
- Ranked research paper snippets
- Citations and references
- System status (retrieving papers, processing, generating response)

Basic UI Workflow

- User enters a research question.
- User clicks **Search / Generate Research**.
- System processes the query through agents.
- Results appear with:
 - Structured summary
 - Key insights
 - Citations

3.1.2 Hardware Interfaces

The system primarily runs on a **standard computing environment**.
The following hardware interfaces are used:

- **User Device**

- Laptop or desktop computer
- Used to access the web interface
- **Server Infrastructure**
 - Cloud or local server running the AI system
 - Handles agent orchestration and query processing
- **Storage Hardware**
 - Local or cloud storage for storing:
 - downloaded PDFs
 - embeddings
 - FAISS index files
- **Networking Hardware**
 - Internet connectivity is required for:
 - accessing the arXiv API
 - calling LLM APIs
 - retrieving research papers

All sensors or hardware components are assumed to provide basic **read/write access through standard system interfaces**.

3.1.3 Software Interfaces

The system interacts with several external software components.

1. arXiv API

Purpose:

- Retrieve research papers related to the user query.

Data exchanged:

- Query keywords
- Paper metadata

- PDF links
-

2. FAISS Vector Database

Purpose:

- Store embeddings of research papers
- Enable semantic similarity search

Operations:

- Add embeddings
 - Retrieve top-k similar vectors
-

3. Large Language Model (LLM)

Purpose:

- Query decomposition
- Research synthesis
- Ranking and reasoning

Example frameworks:

- OpenAI API
 - Local LLM
 - LangChain
-

4. PDF Processing Libraries

Used to extract content from research papers.

Examples:

- PyPDF
 - PDFMiner
 - LangChain document loaders
-

5. Supabase Database

Purpose:

- Store system logs and analytics

Logged data includes:

- User queries
- Response latency
- Number of papers retrieved
- Token usage
- Errors

3.2 Functional Requirements

The functional requirements define the behavior of the system.

F1: Accept Research Query

The system shall allow users to enter a research query through the user interface.

F2: Query Planning

The system shall analyze the user query using a **Planner Agent** and break it into smaller research tasks.

F3: Paper Retrieval

The system shall retrieve relevant research papers from **arXiv** based on the processed query.

F4: Document Download

The system shall download the PDF versions of the retrieved research papers.

F5: Document Processing

The system shall extract text from downloaded PDFs and divide them into smaller chunks for processing.

F6: Embedding Generation

The system shall convert text chunks into vector embeddings using an embedding model.

F7: Vector Storage

The system shall store embeddings in a **FAISS vector database** for efficient similarity search.

F8: Semantic Retrieval

The system shall retrieve the most relevant chunks from the vector database using similarity search.

F9: Document Ranking

The system shall rank retrieved chunks using an LLM-based scoring mechanism.

F10: Research Synthesis

The system shall generate a structured research summary using the **Reasoning Agent**.

F11: Citation Verification

The system shall verify references and reduce hallucination using the **Safety Agent**.

F12: Response Generation

The system shall produce a final research report including:

- summary
 - key findings
 - supporting citations
 - references
-

F13: Logging and Monitoring

The system shall log queries, token usage, latency, and system errors into the Supabase database.

3.3 Use Case Model**Actors:**

- User
- arXiv API
- LLM Service
- Database

Basic diagram representation:

| Multi-Agent RAG System |

User -----> Submit Research Query

User <----- View Research Summary

System -----> Retrieve Papers (arXiv API)

System -----> Generate Reasoning (LLM)

System -----> Store Logs (Supabase DB)

3.3.1 Use Case #1 (use case name and unique identifier – e.g. U1)

Author: Vinay Patil

Purpose:

Allow users to submit a research question and receive a summarized response supported by citations.

Requirements Traceability:

F1, F2, F3, F10, F12

Priority:

High

Preconditions

- User has access to the system interface.
- Internet connection is available.

Postconditions

- Research summary is generated.
- Relevant research papers are cited.
- System logs are recorded.

Actors

Primary Actor:

- User

Secondary Actors:

- arXiv API
 - LLM Service
 - Supabase Database
-

Flow of Events

Basic Flow

1. User enters research query.
 2. System sends query to Planner Agent.
 3. Planner Agent decomposes the query.
 4. Search Agent retrieves papers from arXiv.
 5. System downloads and processes PDFs.
 6. Retrieval Agent finds relevant chunks.
 7. Reasoning Agent synthesizes research summary.
 8. Safety Agent verifies citations.
 9. Final research response is displayed to user.
-

Alternative Flow

- If insufficient papers are retrieved:
 - System expands search keywords
 - Additional papers are fetched

Exceptions

- arXiv API unavailable
- PDF download fails
- LLM API timeout

In such cases, the system returns an **error message to the user**.

Includes

U2 – Retrieve Research Papers

U3 – Generate Research Summary

Notes / Issues

- Performance depends on the number of retrieved papers.
- Response time may increase with large PDFs.

3.3.2 Use Case #2

Author: Sujato Dutta

Purpose:

Retrieve relevant research papers from arXiv based on the user query.

Requirements Traceability:

F3, F4, F5

Priority:

High

Preconditions

- Query has been processed by Planner Agent.
-

Postconditions

- Relevant research papers are downloaded and processed.
-

Flow of Events

1. System sends search query to arXiv API.
 2. API returns relevant paper metadata.
 3. System downloads paper PDFs.
 4. PDFs are processed and stored.
-

If you want, I can also help you create:

- **A clean professional Use Case Diagram**
- **Sequence Diagram**
- **Activity Diagram**

4 Other Non-functional Requirements

4.1 Performance Requirements

The system should process a user query and generate a research response within a reasonable time under normal operating conditions. On average, the system should return a response within **20–30 seconds**, depending on the number of research papers retrieved and processed.

The system should also be capable of retrieving and processing multiple research papers in a single query while maintaining acceptable response time and system stability.

4.2 Safety and Security Requirements

The system should ensure that generated responses are based on retrieved research documents to reduce hallucinations. A **Safety Agent** is used to verify citations and ensure that the generated answers are grounded in reliable sources.

Communication with external services such as the **arXiv API** and language model APIs should use secure connections. The system should also prevent unauthorized access to the system and ensure that user queries and retrieved data are handled securely.

4.3 Software Quality Attributes

Reliability: The system should consistently retrieve research papers and generate responses without failures. Error handling mechanisms should be implemented to manage issues such as API failures or missing documents.

Maintainability: The system uses a **modular multi-agent architecture**, allowing individual agents (Planner, Search, Retrieval, Reasoning, etc.) to be modified or updated without affecting the entire system.

Scalability: The system should be designed so that additional agents or data sources can be integrated in the future, allowing the system to support more complex research tasks.

5 Other Requirements

The Agentic System is integrated with a Supabase database to systematically log and store operational metrics generated during user interactions. These logs include user queries, system response status, tokens consumed by the language model, latency of each request, and the number of chunks and research papers retrieved during the retrieval process. In addition, the system records hallucination flags when responses may contain unsupported information, along with any execution errors encountered during processing.

Maintaining these logs is critical for monitoring the system in a production environment. The collected data enables the development team to evaluate system performance, identify potential failures, and optimize retrieval efficiency and response generation. Furthermore, tracking these metrics supports continuous improvement of system reliability, accuracy, and resource utilization.

The architecture is designed to support future enhancements. Potential extensions include integration with additional academic research repositories to broaden the system's knowledge sources, incorporation of persistent vector databases for efficient large-scale similarity search, and the addition of advanced reasoning capabilities to enable more complex multi-step research analysis.

Appendix A – Data Dictionary

Constants

Name	Value	Description	Related Requirements
MAX_PAPERS_RETRIEVED	10	Maximum number of arXiv papers fetched per query	F3
MAX_RESPONSE_TIME_SEC	30	Maximum acceptable response time (seconds)	4.1
CHUNK_SIZE	~500 tokens	Size of each text chunk extracted from PDFs	F5
TOP_K_RETRIEVAL	5	Number of top similar vectors retrieved from FAISS	F8
EMBEDDING_DIMENSIONS	768 (model-dependent)	Vector size used for FAISS indexing	F6, F7
LLM_MAX_TOKENS	1000	Max tokens allowed per LLM API call	F9, F10

State Variables

Variable	Type	Possible States	Description	Related Requirements
query_status	Enum	IDLE, PLANNING, SEARCHING, INGESTING, RETRIEVING, RERANKING, REASONING, VALIDATING, COMPLETE, ERROR	Tracks the current pipeline stage of a user query	F1–F12
paper_download_status	Enum	PENDING, DOWNLOADING, SUCCESS, FAILED	Status of each PDF download attempt	F4
hallucination_flag	Boolean	TRUE, FALSE	Set by Safety Agent if response contains unverified claims	F11
faiss_index_status	Enum	EMPTY, BUILDING, READY	State of the FAISS vector index for the current session	F7
llm_api_status	Enum	AVAILABLE, TIMEOUT, ERROR	Tracks LLM API availability during a session	F9, F10
arxiv_api_status	Enum	AVAILABLE, UNAVAILABLE, RATE_LIMITED	Tracks arXiv API availability	F3
session_state	Enum	ACTIVE, EXPIRED, TERMINATED	Lifecycle state of a user session	F1, F13

Inputs

Name	Type	Source	Description	Related Requirements
user_query	String	User (Web UI)	Natural language research question entered by user	F1
search_keywords	String[]	Planner Agent	Decomposed keywords extracted from user query	F2
arxiv_paper_metadata	JSON	arXiv API	Paper title, authors, abstract, PDF URL returned by arXiv	F3
pdf_file	Binary (PDF)	arXiv API	Downloaded PDF document of a research paper	F4
text_chunks	String[]	Ingestion Agent	Extracted and split text segments from a PDF	F5
query_embedding	Float[]	Embedding Model	Vector representation of the user query for similarity search	F8

Outputs

Name	Type	Destination	Description	Related Requirements
research_summary	String	Web UI	Structured summary generated by Reasoning Agent	F10, F12
key_findings	String[]	Web UI	Bullet-point key insights extracted from retrieved papers	F12
citations	String[]	Web UI	List of paper references supporting the generated response	F11, F12
ranked_chunks	String[]	Reasoning Agent	Top-ranked text chunks after LLM-based reranking	F9

hallucination_report	JSON	Safety Agent	Report indicating whether any claims are unverified	F11
error_message	String	Web UI	Human-readable error shown when pipeline stage fails	F12
system_log_entry	JSON	Supabase DB	Logged record containing query, latency, token usage, errors	F13

Operations / Process Variables

Name	Type	Description	Related Requirements
query_decomposition	Operation	Planner Agent breaks user query into focused sub-queries/keywords	F2
semantic_similarity_search	Operation	FAISS compares query embedding against stored chunk embeddings	F8
llm_reranking_score	Float (0.0–1.0)	Relevance score assigned to each chunk by the LLM reranker	F9
embedding_vector	Float[]	Numerical vector representation of a text chunk stored in FAISS	F6, F7
token_usage_count	Integer	Number of tokens consumed per LLM API call	F13
response_latency_ms	Integer	Time (ms) from query submission to final response delivery	F13, 4.1
papers_retrieved_count	Integer	Number of papers fetched and processed for a given query	F3, F13
chunks_retrieved_count	Integer	Number of chunks returned from FAISS for a given query	F8, F13

Appendix B - Group Log

Member Section Assignments

Member	Roll Number	Section Assigned
Mude Sai Abhinav Chauhan	SE23UCSE198	Section 1 – Introduction (Document Purpose, Scope, Audience, Definitions, Conventions, References)
Anish Boppidi Reddy	SE23UCSE042	Section 2.1 & 2.2 – Product Overview & Functionality
Aishwarya Vodnala	SE23UCSE187	Section 2.3 & 2.4 – Design Constraints & Assumptions; Document Compilation & Formatting
Vinay Patil	SE23UCSE186	Section 3.1 – External Interface Requirements; Use Case #1
Tejas Pothu	SE23UCSE175	Section 3.2 – Functional Requirements (F1–F13)
Siddharth Uppala	SE23UCSE241	Section 4 – Non-Functional Requirements (Performance, Safety, Quality)
Supreeth Kunaparaju	SE23UCSE098	Section 5 & Appendix A – Other Requirements & Data Dictionary
Sujato Dutta	SE23UARI167	Section 3.3 – Use Case Model; Use Case #2

Meeting Minutes

Meeting 1 — Project Kickoff

Date: 9 February 2026

Mode: In-person

Attendees: All 8 members

Agenda:

- Understand the SRS assignment requirements and IEEE format
- Decide on the project idea

Discussion:

- The team agreed on building a Multi-Agentic RAG AI Research Scientist system
- The choice was motivated by the group's interest in LLMs, RAG pipelines, and multi-agent architectures
- Abhinav volunteered to lead document compilation and version control
- Agreed to use WhatsApp for day-to-day communication and coordination

Decisions Made:

- Project topic confirmed: Multi-Agentic RAG Research System
- IEEE SRS template to be followed strictly
- Each member to independently research their assigned area before the next meeting

Action Items:

Member	Task
All members	Study IEEE SRS template and understand their assigned section
Abhinav	Set up shared document and assign sections
Tejas & Supreeth	Research RAG architecture, FAISS, and arXiv API
Anish & Aishwarya	Research multi-agent system design patterns

Meeting 2 — Research Review & Section Planning

Date: 9 February 2026

Mode: In-person

Attendees: All 8 members

Agenda:

- Share research findings on RAG, FAISS, and arXiv
- Finalize system architecture and agent pipeline
- Plan individual section content

Discussion:

- Tejas and Supreeth presented findings on FAISS vector indexing and RAG pipelines
- Anish walked through multi-agent design — 7-agent pipeline was finalized (Planner, Search, Ingestion, Retrieval, Reranking, Reasoning, Safety)
- Vinay outlined the use case structure and proposed two main use cases
- Sujato confirmed Use Case #2 scope (paper retrieval flow)
- The group discussed functional requirements — 13 functional requirements (F1–F13) were identified and listed

Decisions Made:

- 7-agent pipeline architecture confirmed
- Supabase chosen for logging and monitoring (F13)
- FAISS confirmed as the vector database (session-scoped, ephemeral)
- Groq API confirmed as the LLM inference backend

Action Items:

Member	Task
All members	Begin drafting their assigned sections
Abhinav	Draft Section 1 and set up document structure
Vinay	Draft Use Case #1 with full flow of events
Siddharth	Begin non-functional requirements section

Meeting 3 — Draft Review & Integration

Date: ~03 March 2026 **Mode:** In-person + WhatsApp coordination **Attendees:** All 8 members

Agenda:

- Review all drafted sections
- Identify gaps and inconsistencies
- Integrate all sections into one document

Discussion:

- Each member presented their drafted section for group review
- Minor inconsistencies in terminology were identified (e.g., "Ingestion Agent" vs "Document Agent") — standardized to match the README
- Data Dictionary (Appendix A) was discussed; Supreeth took ownership of completing it
- Appendix B (Group Log) structure was agreed upon
- Siddharth's non-functional requirements section was reviewed and approved with minor edits

Decisions Made:

- All agent names standardized to match the system architecture diagram in the README
 - Response time target set at 20–30 seconds (Section 4.1)
 - Final document to be submitted by 09 March 2026
-

Meeting 4 — Final Review & Submission Preparation

Date: ~07 March 2026 **Mode:** WhatsApp **Attendees:** All 8 members (via group chat)

Agenda:

- Final review of the complete SRS document
- Confirm submission readiness

Discussion:

- Abhinav shared the final merged document over WhatsApp for review
- All members reviewed and confirmed their sections were accurate
- Minor formatting fixes were made (font, spacing, table alignment)
- Group confirmed all 13 functional requirements were traceable to use cases

Decisions Made:

- Document approved for submission
 - Abhinav designated as the member to submit the final document
-

Summary of Group Effort

Activity

Details

Total Meetings	4 (3 in-person, 1 via WhatsApp)
Communication Channel	WhatsApp group for daily updates and file sharing
Research Activities	RAG pipelines, FAISS vector search, arXiv API, multi-agent architectures, Groq API
Writing Approach	Sections divided and written independently, then integrated
Review Process	Peer review during Meeting 3; final review during Meeting 4
Document Versions	Version 1.0 (initial and final draft)
Submission Date	16 March 2026
